

Catalyst Optimizer vs AQE

1 Execution Time

- **Catalyst Optimizer** → Works **before execution** (compile time) -----(imp point)

It works during the **compile time** phase to optimize the logical plan.

It can:

- Apply **Predicate Pushdown**
- Perform **Column Pruning**
- Do **Constant Folding**
- Reorder **Filters and Projections**
- Simplify expressions

- **AQE** → Works **during execution** (runtime) -----(imp point)

AQE dynamically **changes the execution plan during runtime**.

It can:

- Switch join strategies
- Optimize shuffle partitions
- Handle data skew
- Reduce unnecessary shuffles

Example:

If a table becomes small at runtime → Spark may switch to **Broadcast Join**.

2 Optimizes Which Plan?

- **Catalyst** → Optimizes **Logical Plan**
- **AQE** → Optimizes **Physical Plan**

3 Type of Optimization

- **Catalyst** → Rule-based + Cost-based optimization
- **AQE** → Runtime adaptive optimization based on actual data

4 Data Awareness

- **Catalyst** → Uses statistics (if available), not real-time data
- **AQE** → Uses real shuffle output data

5 Join Strategy Changes

- **Catalyst** → Chooses join type before execution
- **AQE** → Can change join type at runtime (e.g., Sort Merge → Broadcast)

6 Shuffle Partition Handling

- **Catalyst** → Cannot change shuffle partitions after planning
- **AQE** → Can reduce shuffle partitions dynamically

7 Skew Handling

- **Catalyst** → ❌ Does not handle data skew
- **AQE** → ✅ Handles skewed partitions

8 Performance Impact

- **Catalyst** → Improves query efficiency before execution
- **AQE** → Improves performance dynamically during execution

9 Configuration Required?

- **Catalyst** → Enabled by default (core Spark engine)
- **AQE** → Needs to be enabled: `spark.conf.set("spark.sql.adaptive.enabled", "true")`

10 Example Scenario

- **Catalyst**: Pushes filter before join
- **AQE**: Converts large join into broadcast join at runtime